

fpga finals

Astelor

Here's what I built

- A snake game with LTM module

Flow Summary	
Flow Status	Successful - Fri Jun 05 00:03:24 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Full Version
Revision Name	fpga_snake_game
Top-level Entity Name	DE2_70_LTM_Ephoto
Family	Cyclone II
Device	EP2C70F896C6
Timing Models	Final
Total logic elements	2,421 / 68,416 (4 %)
Total combinational functions	2,314 / 68,416 (3 %)
Dedicated logic registers	611 / 68,416 (< 1 %)
Total registers	611
Total pins	534 / 622 (86 %)
Total virtual pins	0
Total memory bits	20,480 / 1,152,000 (2 %)
Embedded Multiplier 9-bit elements	2 / 300 (< 1 %)
Total PLLs	0 / 4 (0 %)



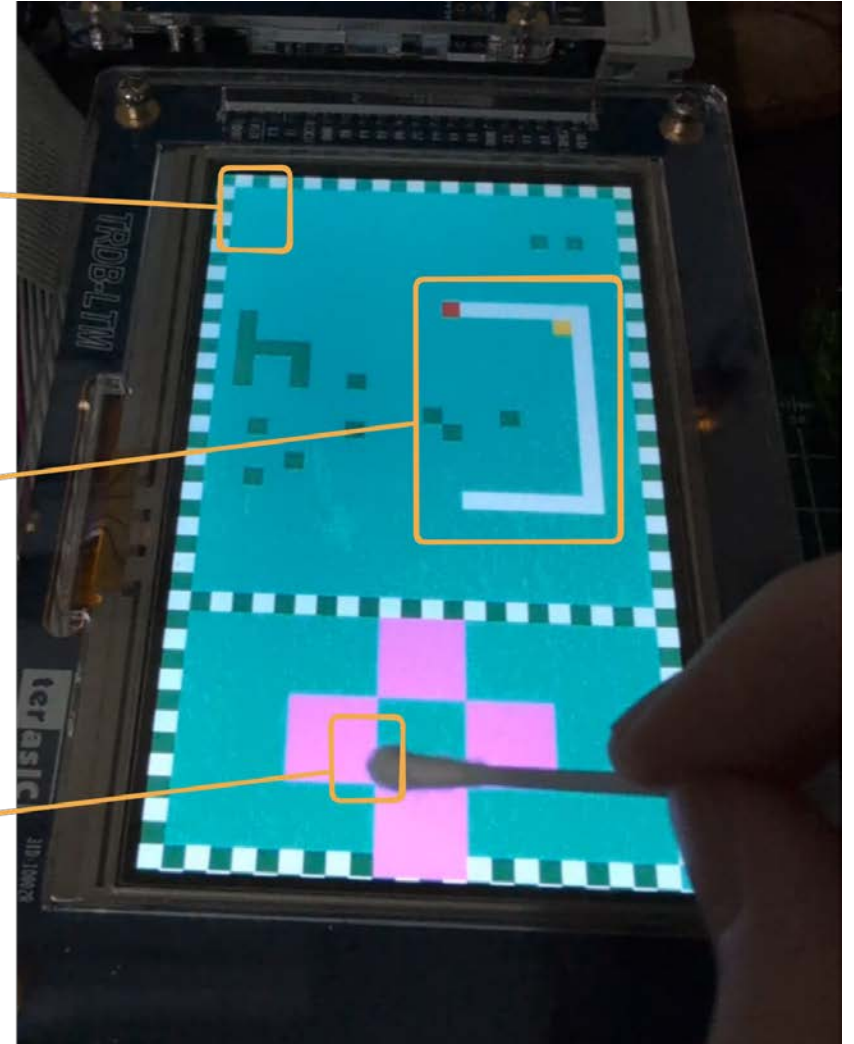
Core components

- also the outline of this presentation

pixelated display

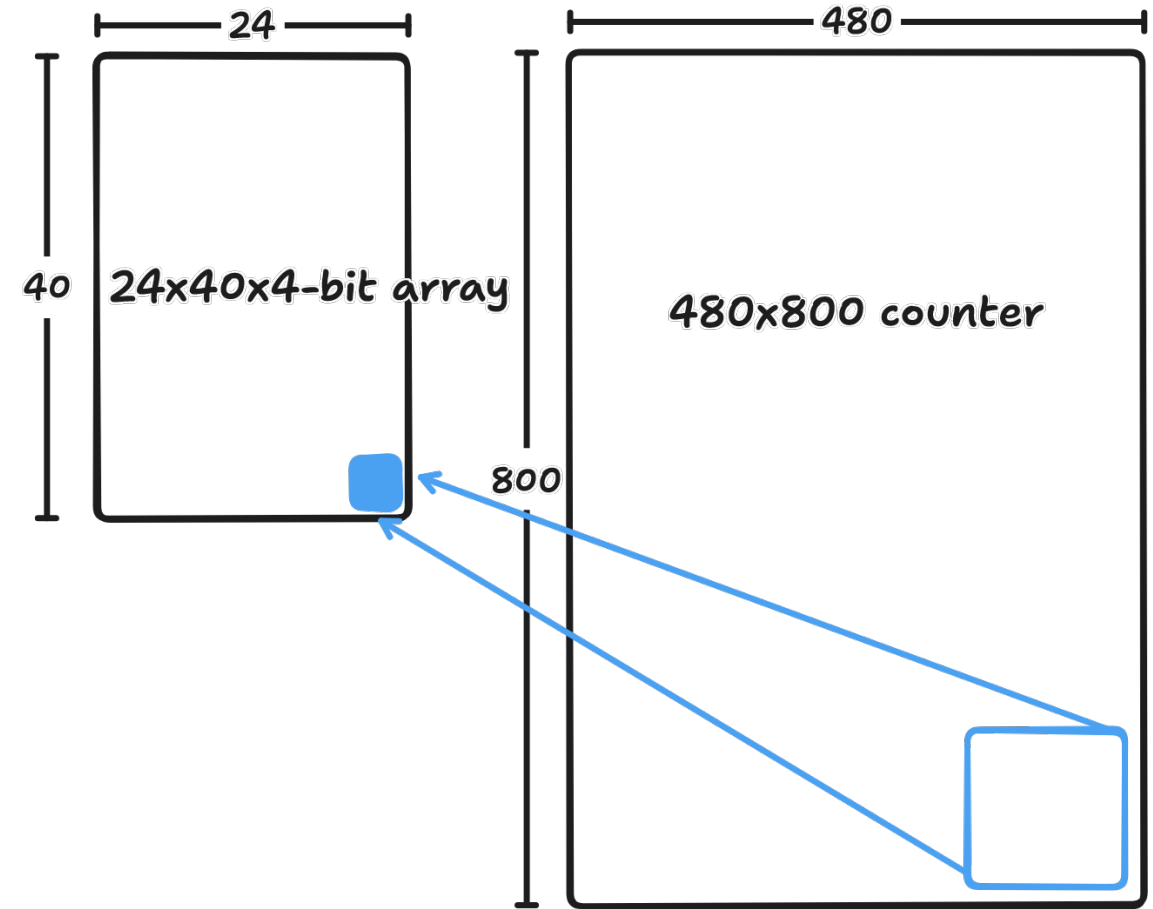
snake game logic

touch screen coordinate



Pixelated display

- Only need 24x40 display of limited colors
- → 40x24x4-bit array to store the data needed
 - = 3840 bits
- A counter for the physical pixels on LCD panel.
- f is a map. Convert counter to x & y coordinates on the 4-bit array



$f: N \rightarrow N \times N, f(\text{counter}) = (x, y)$
 $x = (\text{counter} / x_block_length) \% x_resolution$
 $y = (\text{counter} / (x_block_length \times y_block_length \times x_resolution)) \% y_resolution$

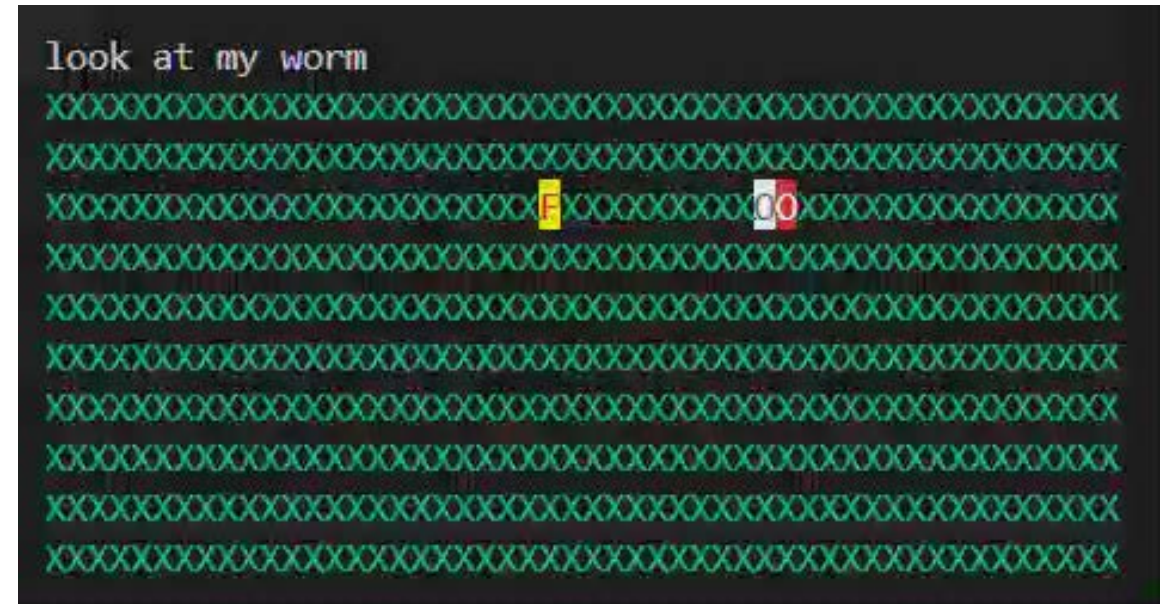
Snake game logic

I built the game in C++ first.

Key features of this build

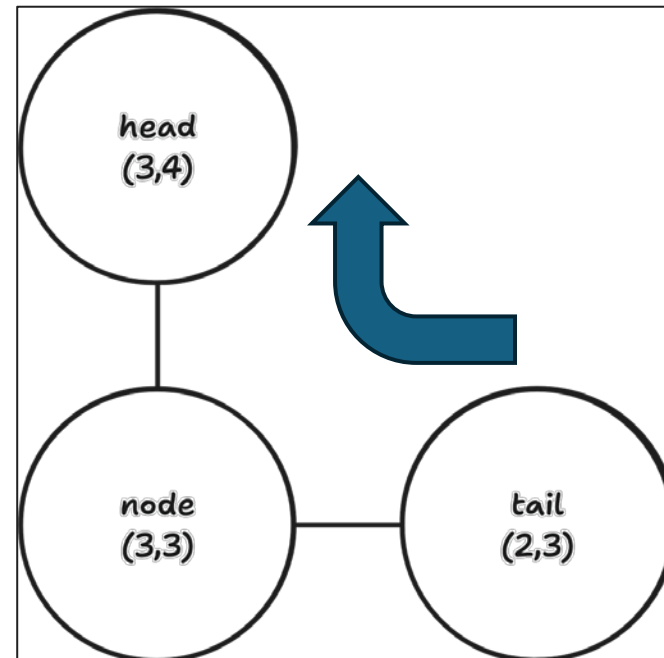
- ASCII terminal display (curses library on Linux)
- Threading for timers and concurrency
- mutex lock for share resource protection

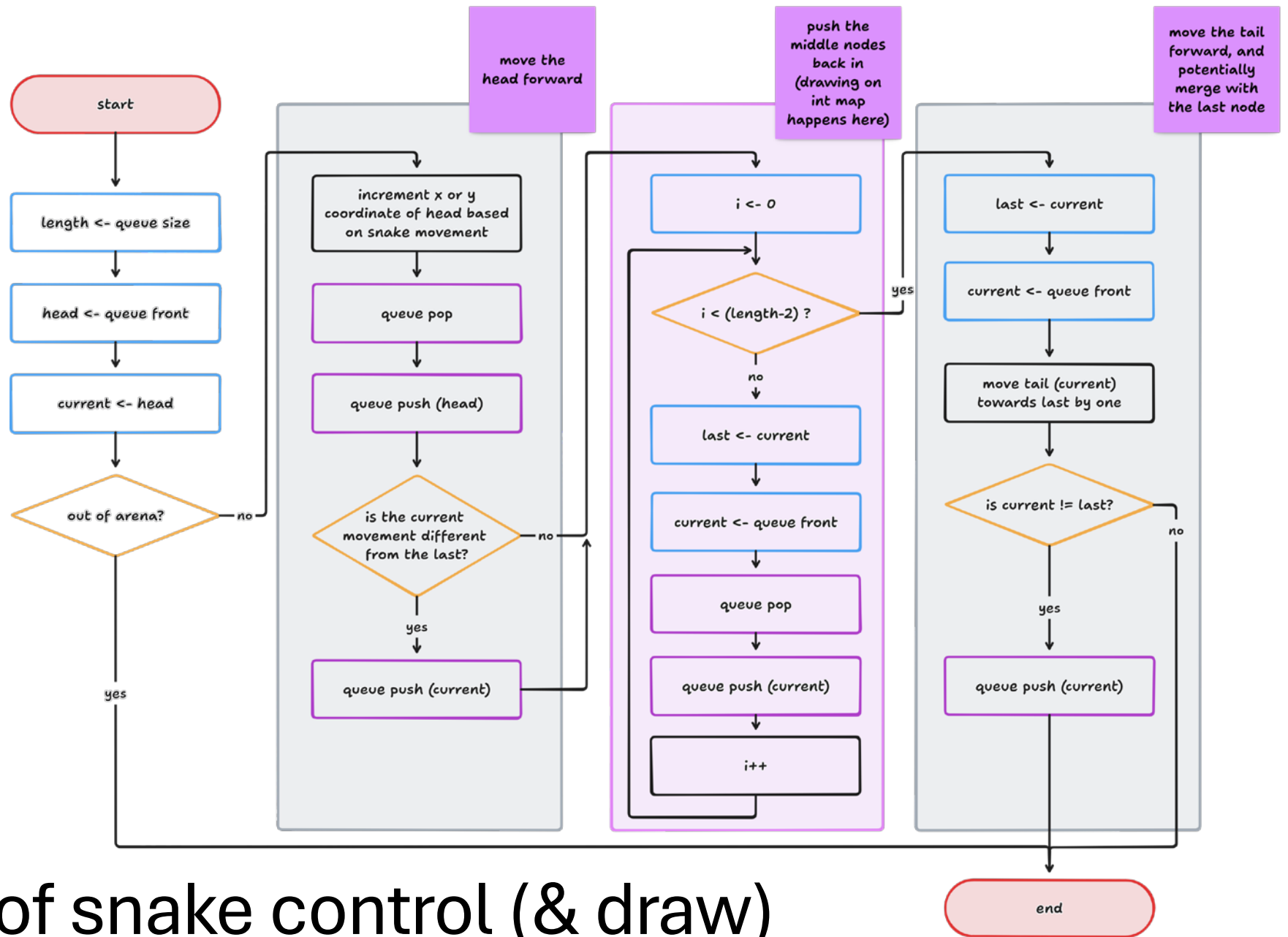
A reference project for Verilog



Snake location data

- The snake is a **queue**
- Any turn in the snake is a **node**
- All the nodes are in absolute coordinate
- To draw the snake on the map, take two nodes and trace the distance back.

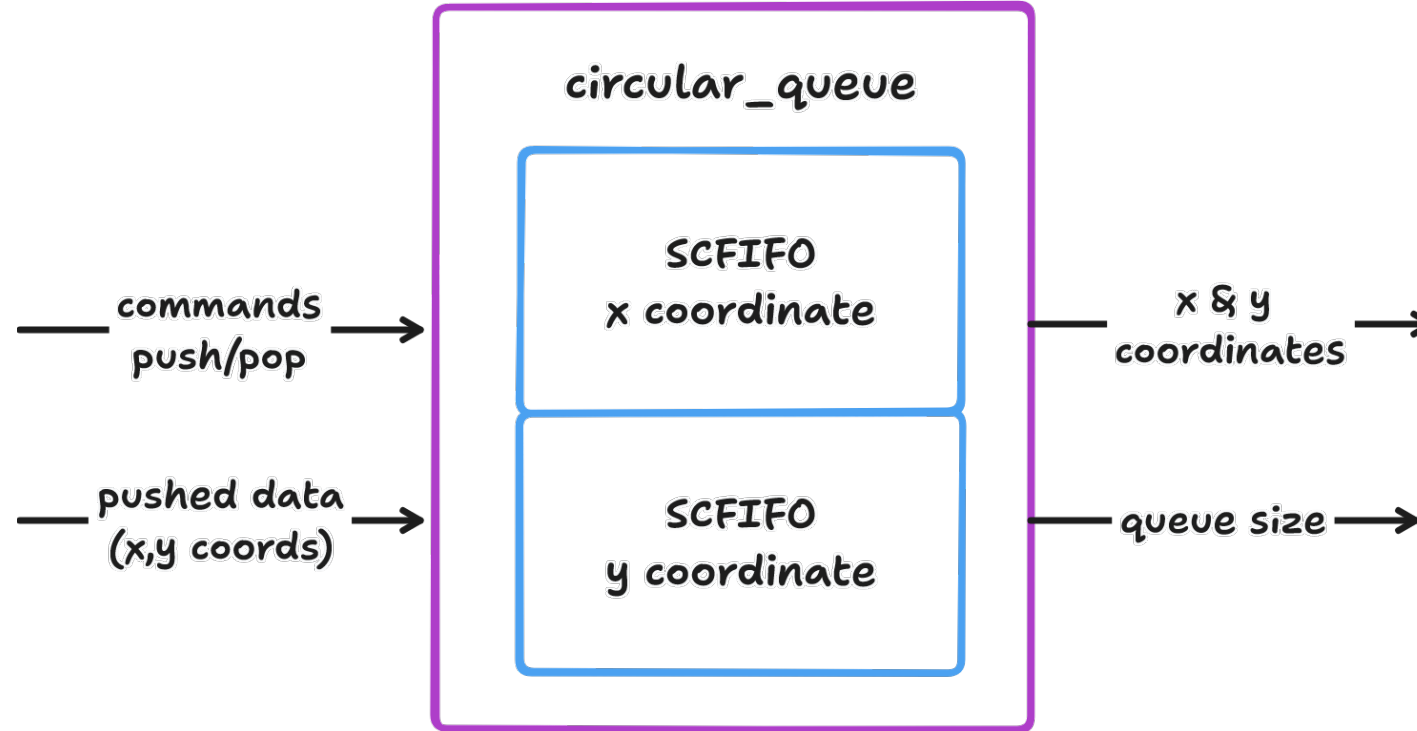




Flowchart of snake control (& draw)

Circular queue

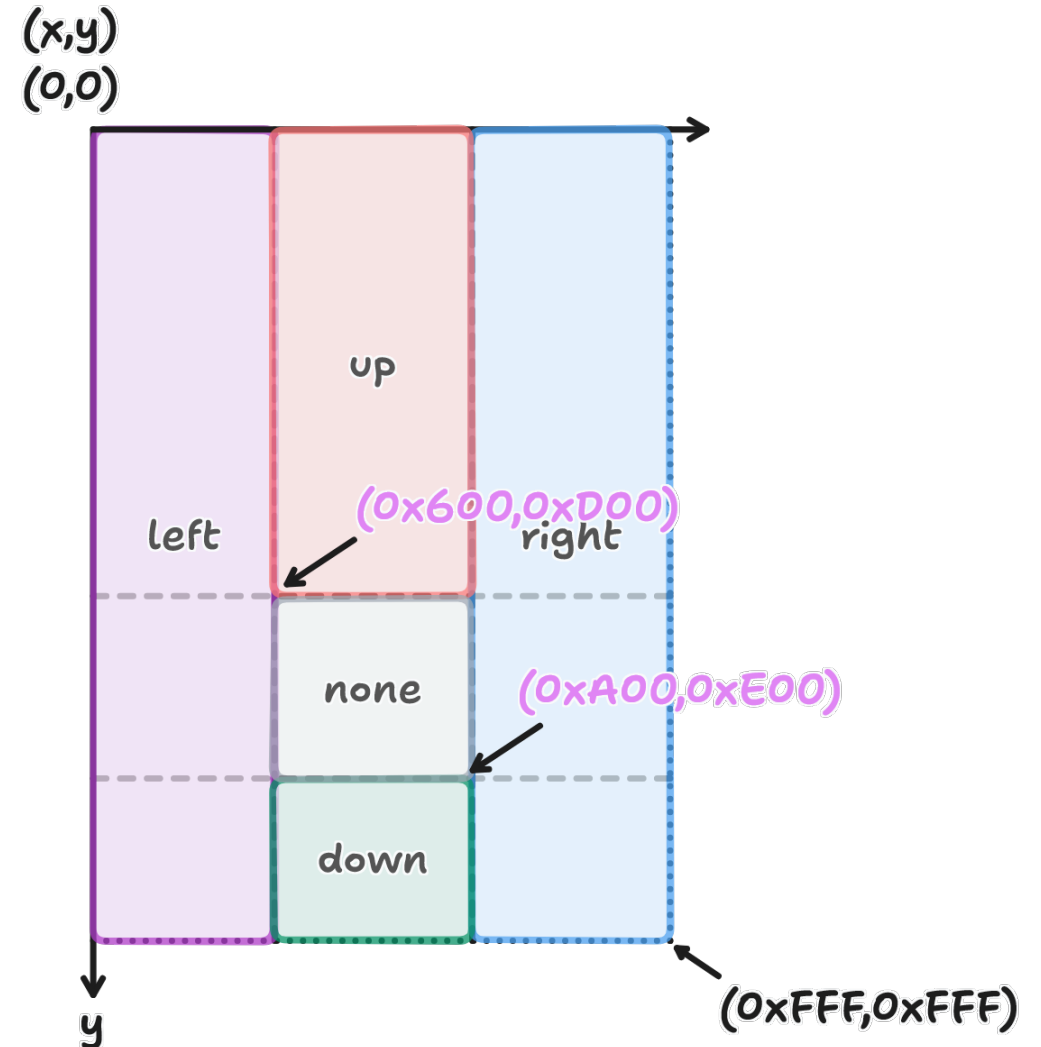
- A wrapper around two SCFIFOs



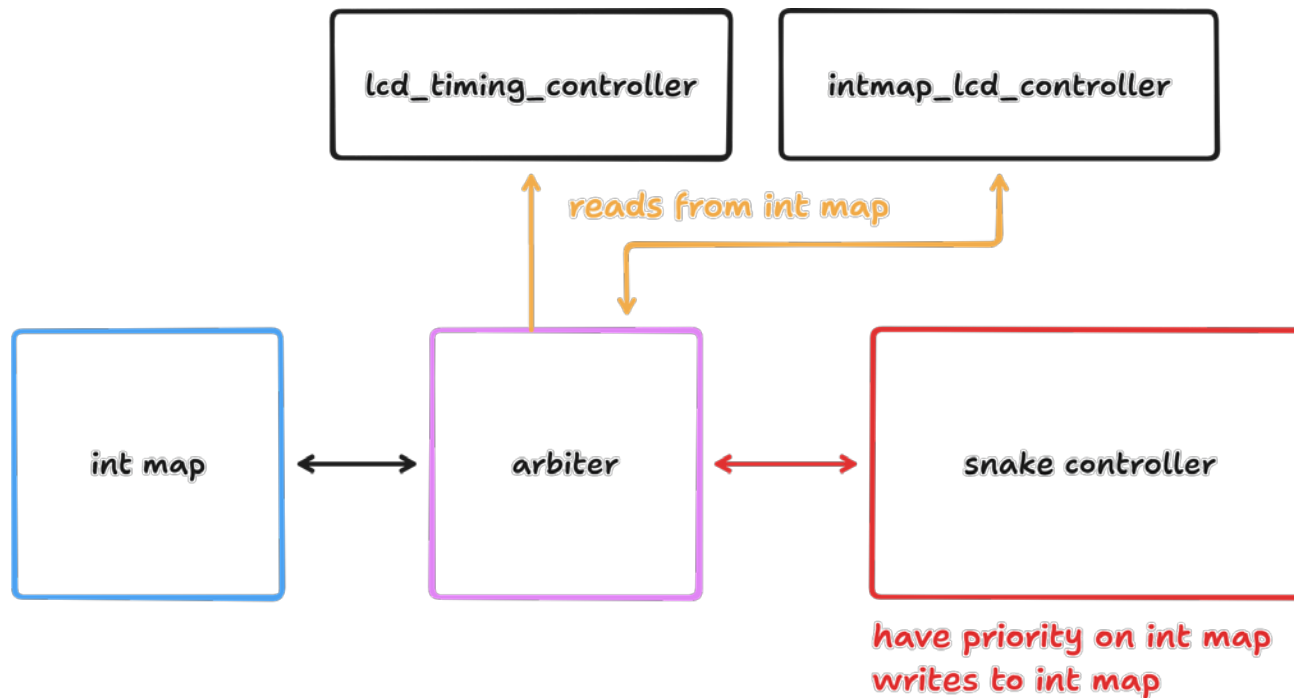
Coordinate checking

- convert coordinate into movement

```
always @(posedge iCLK or negedge iRST_n) begin
    if(!iRST_n) begin
        mMOVEMENT <= NONE;
        mNEW_MOVE <= 0;
    end
    else if(mtrig) begin
        mNEW_MOVE <= 1;
        if( (mY_COORD < CTRL_Y1) && (mX_COORD > CTRL_X1) && (mX_COORD < CTRL_X2))
            mMOVEMENT <= MOVE_UP;
        else if( (mY_COORD > CTRL_Y2) && (mX_COORD > CTRL_X1) && (mX_COORD < CTRL_X2))
            mMOVEMENT <= MOVE_DOWN;
        else if( (mX_COORD < CTRL_X1))
            mMOVEMENT <= MOVE_LEFT;
        else if( (mX_COORD > CTRL_X2))
            mMOVEMENT <= MOVE_RIGHT;
        else
            mMOVEMENT <= NONE;
    end
    else begin
        mNEW_MOVE <= 0;
    end
end
```

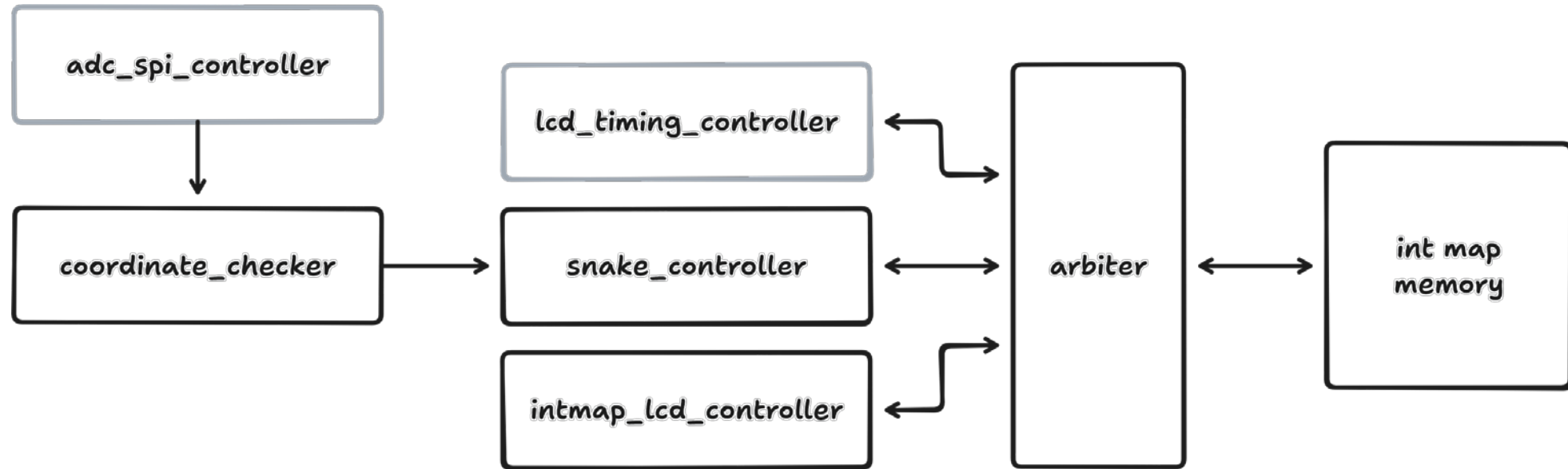


Shared resource managing



```
always @(posedge iCLK or negedge iRST_n) begin
    if(!iRST_n) begin
        rState <= IDLE;
    end else begin
        /*==== Default Signals ====*/
        rState <= rState;
        /*==== State Machine =====*/
        case (rState)
            IDLE: begin // give RAM to intmap_lcd_controller
                oWriteEnable <= 0;
                oMemAddr <= iLcdAddr;
                oLcdData <= iMemReadData;
                if(iSnakeStatus == STAT_WRITE) begin
                    rState <= SNAKE_WRITE;
                end
                else if(iSnakeStatus == STAT_CHECK)begin
                    rState <= SNAKE_CHECK;
                end
            end
            SNAKE_WRITE: begin
                oWriteEnable <= 1;
                oMemAddr <= iSnakeAddr;
                oSnakeData <= iMemReadData;
                oMemWriteData <= iSnakeData;
                if(iSnakeStatus == STAT_IDLE) begin
                    rState <= IDLE;
                end
                else if(iSnakeStatus == STAT_CHECK)begin
                    rState <= SNAKE_CHECK;
                end
            end
            SNAKE_CHECK: begin // only read ...
            end
        endcase
    end
end
```

Architecture



Food generation in Verilog

- Because \$random is not synthesizable
- Using modular arithmetic *seem* random enough

```
always @(posedge iCLK or negedge iRST_n) begin
    if(!iRST_n) begin
        rFoodX <= 25;
        rFoodY <= 7;
    end
    else if(is_food)begin
        rFoodX <= (rHeadX * rFoodX) % (SNAKE_X-SNAKE_X_OFFSET-2) + SNAKE_X_OFFSET + 1;
        rFoodY <= (rHeadY * rFoodX) % (SNAKE_Y);
    end
end
```

Map generation for int map

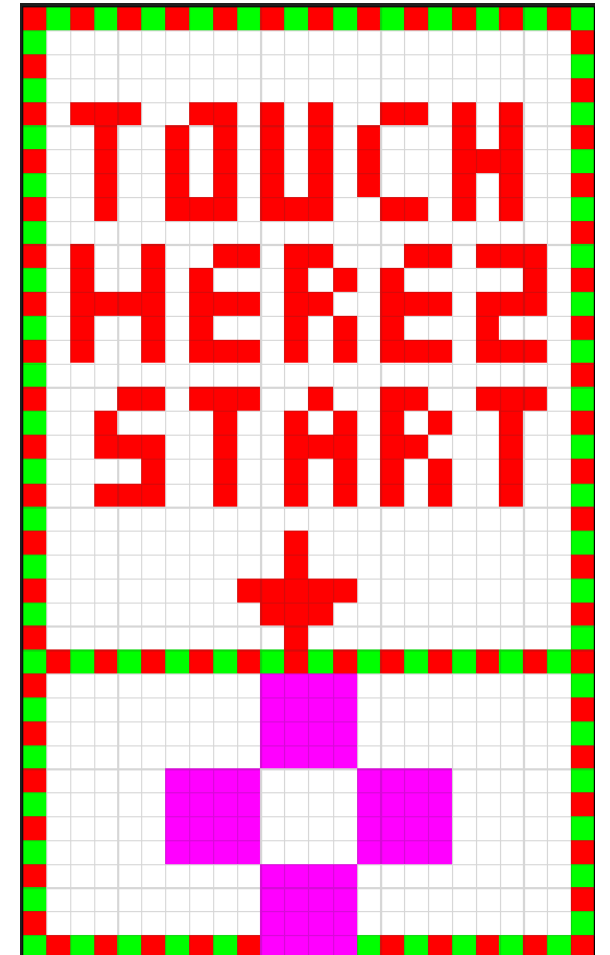
```
R,G,B,w,h = get_image_hex("map40x24.png")

colors = [
    [0,"FF","FF","FF"], # grass (green)
    [1,"00","FF","00"], # body (white)
    # [2,"00","00","00"], # head (red)
    # [3,"00","00","00"], # food (yellow)
    [4,"FF","00","FF"], # control (pink)
    [5,"FF","00","00"] # dark green
]

fd = open("map_new.hex","w",newline='\n')
for i in reversed(range(w*h)):
    temp = 0
    for k in colors:
        if ((R[i] == k[1]) and (G[i] == k[2]) and (B[i] == k[3])):
            temp = k[0]
            break
    print(temp,file=fd)
fd.close()
```

✓ 0.0s

Python



github repositories

- <https://github.com/astelor/fpga-snake-game>
 - (I'll make it public when I am done tidying it)
- <https://github.com/astelor/snake-game>

Thanks for listening

:P